Strings in TypeScript

A **string** in TypeScript is a sequence of characters used to represent text. It can be declared using:

1. **Single quotes** 'like this'
2. **Double quotes** "like this"
3. **Backticks (Template literals)** `like this` – useful when embedding variables inside strings.

```
let str1: string = 'Single quote';

let str2: string = "Double quote";

let str3: string = `Backtick`;
```

Why use Backticks?

Backticks allow **template literals**, which support **string interpolation** using ${}:

```
let num: number = 10;

console.log(`Number is: ${num}`); // ■ Correct way
```

+ 'Number is: ${num}' or "Number is: ${num}" won't work as expected.

Common String Methods

```
let greeting: string = "Hello, TypeScript!";
```

1. **.length** – Get the number of characters
   greeting.length → 18
2. **.toUpperCase() / .toLowerCase()** – Change case
   greeting.toUpperCase() → HELLO, TYPESCRIPT!
   greeting.toLowerCase() → hello, typescript!
3. **.charAt(index) / .indexOf(substring)** – Get a character or find a substring
   greeting.charAt(4) → o
   greeting.indexOf("Type") → 7
4. **.substring(start, end)** – Extract a part of the string
   greeting.substring(7, 11) → Type
5. **.includes(substring)** – Check if a substring exists
   greeting.includes("Script") → true

6. **.startsWith() / .endsWith()** – Check start or end of string
   greeting.startsWith("Hello") → true
   greeting.endsWith("!") → true
7. **.replace(old, new)** – Replace part of a string
   greeting.replace("TypeScript", "World") → Hello, World!
8. **.split(delimiter)** – Split into array
   greeting.split(" ") → ['Hello,', 'TypeScript!']
9. **.trim() / .trimStart() / .trimEnd()** – Remove extra spaces
   Removes spaces from start, end, or both.

String Immutability

Strings are **immutable** – once created, they can't be changed. Methods return **new strings**:

```
let original = "Hello";

let modified = original.concat(", World!");

// original is still "Hello"
```

Multi-line Strings

Using backticks allows strings to span multiple lines:

```
let multiLine = `Line one

Line two

Line three`;
```

String Examples

Example 1: Declaring strings using quotes

A string can be written using **single quotes**, **double quotes** or **backticks**. The index of a character in a string starts with 0.

```
let str1: string = 'This is a string with single quote';
let str2: string = "This is a string with double quote";
let str3: string = `This is a string with back tick`;
console.log(str1);
console.log(str2);
console.log(str3);
```

Output:

```
This is a string with single quote
This is a string with double quote
This is a string with back tick
```

Example 2: Template string using backtick

${ } works only inside **backticks**. Inside single or double quotes it is printed as normal text.

```
let num: number = 10;
console.log("Number is:", num);   // Valid
console.log("Number is: ${num}");   // Printed as text
console.log('Number is: ${num}');   // Printed as text
console.log(`Number is: ${num}`);   // Valid
console.log(`Number plus 5 is: ${num + 5}`);   // Expressions also work
```

Output:

```
Number is: 10
Number is: ${num}
Number is: ${num}
Number is: 10
Number plus 5 is: 15
```

Example 3: length property

length is a property (not a method), so it is written without parentheses. Spaces, commas and symbols are also counted.

```
let str: string = "Hello, TypeScript!";
console.log("Length of a string:", str.length);
```

Output:

```
Length of a string: 18
```

Example 4: toUpperCase() and toLowerCase()

Both methods return a **new string**. The original string is not changed.

```
let str: string = "Hello, TypeScript!";
console.log("Upper case:", str.toUpperCase());
console.log("Lower case:", str.toLowerCase());
console.log("Original string:", str);
```

Output:

```
Upper case: HELLO, TYPESCRIPT!
Lower case: hello, typescript!
Original string: Hello, TypeScript!
```

Example 5: charAt() and indexOf()

charAt(index) returns the character at the given index. **indexOf(text)** returns the index where the text starts, or **-1** if it is not found.

```
let str: string = "Hello, TypeScript!";
console.log("Character at 4th index:", str.charAt(4));
console.log("Index of Type:", str.indexOf("Type"));
console.log("Index of abc:", str.indexOf("abc"));
```

Output:

```
Character at 4th index: o
Index of Type: 7
Index of abc: -1
```

Example 6: substring()

The starting index is **included** and the ending index is **exclusive** (not included). If the ending index is not given, it extracts up to the end of the string.

```
let str: string = "Hello, TypeScript!";
console.log(str.substring(0, 5));   // index 0 to 4
console.log(str.substring(7));   // index 7 to the end
```

Output:

```
Hello
TypeScript!
```

Example 7: includes()

Returns **true** or **false**. The check is **case sensitive**.

```
let str: string = "Hello, TypeScript!";
console.log(str.includes("abc"));
console.log(str.includes("TypeScript"));
console.log(str.includes("!"));
console.log(str.includes("TYPESCRIPT"));   // different case
```

Output:

```
false
true
true
false
```

Example 8: startsWith() and endsWith()

Both methods return **true** or **false** and are case sensitive.

```
let str: string = "Hello, TypeScript!";
console.log("starts with:", str.startsWith("Hello"));
console.log("ends with:", str.endsWith("!"));
console.log("ends with abc:", str.endsWith("abc"));
```

Output:

```
starts with: true
ends with: true
ends with abc: false
```

Example 9: replace()

Replaces only the **first match** and returns a **new string**. The original string is not changed.

```
let str: string = "Hello, TypeScript!";
console.log("Replaced string:", str.replace("TypeScript", "World"));
console.log("Original string:", str);
```

Output:

```
Replaced string: Hello, World!
Original string: Hello, TypeScript!
```

Example 10: split()

Breaks the string into parts based on a **delimiter** and returns an **array**. The delimiter itself is not part of the result.

```
let str: string = "Hello, TypeScript!";
let words: string[] = str.split(" ");
console.log("After splitting string:", words);

let myString: string = "abc@gmail.com,xyzabc";
let arr = myString.split(",");
console.log("email:", arr[0]);
console.log("password:", arr[1]);
```

Output:

```
After splitting string: [ 'Hello,', 'TypeScript!' ]
email: abc@gmail.com
password: xyzabc
```

Example 11: trim(), trimStart() and trimEnd()

trim() removes spaces from both ends, **trimStart()** from the start only and **trimEnd()** from the end only. The text is printed inside [ ] so that the spaces can be seen. (trimStart() and trimEnd() need compiler target ES2019 or later.)

```
let myString: string = "   welcome to typescript   ";
console.log(`Original string:   [${myString}]`);
console.log(`trim string:      [${myString.trim()}]`);
console.log(`trimStart string:  [${myString.trimStart()}]`);
console.log(`trimEnd string:    [${myString.trimEnd()}]`);
```

Output:

```
Original string:   [   welcome to typescript   ]
trim string:      [welcome to typescript]
trimStart string:  [welcome to typescript   ]
trimEnd string:    [   welcome to typescript]
```

Example 12: concat()

Joins two or more strings and returns a **new string**. **No space** is added automatically.

```
let str1: string = "welcome";
let str2: string = "to typescript";
let str3: string = " and javascript";
console.log(str1.concat(str2));   // no space is added
console.log(str1 + str2);   // + gives the same result
console.log(str1.concat(str2).concat(str3));   // chaining
console.log(str1.concat(" ", str2, str3));   // several values, with a space
```

Output:

```
welcometo typescript
welcometo typescript
welcometo typescript and javascript
welcome to typescript and javascript
```

Example 13: String immutability

Strings are **immutable** (cannot be changed). Every string method returns a **new string**, so store the returned value if you want to keep it.

```
let num: number = 10;
let res = num + 5;
console.log(res);   // 15
console.log(num);   // 10 (not changed)

let str1: string = "welcome";
let modifiedString = str1.concat(" to typescript");
console.log(str1);   // welcome
console.log(modifiedString);   // welcome to typescript

str1.toUpperCase();   // returned value is thrown away
console.log(str1);   // still "welcome"
str1 = str1.toUpperCase();   // store the returned value
console.log(str1);   // WELCOME
```

A single character also cannot be changed using its index. For example, city[0] = "B" gives the compiler error: Cannot assign to '0' because it is a read-only property.

Output:

```
15
10
welcome
welcome to typescript
welcome
WELCOME
```

Example 14: Multiline string using backtick

A backtick string can span **multiple lines**. The spaces used for indentation inside the backticks also become part of the string.

```
let multiline: string = `welcome
      to typescript`;
console.log(multiline);

let multiline2: string = `welcome
to typescript`;
console.log(multiline2);

console.log("welcome\nto typescript");   // \n is the new line character
```

Output:

```
welcome
      to typescript
welcome
to typescript
welcome
to typescript
```

Solutions – String Methods

Solution 1: length property

Task: Find the number of characters in the string "Hello World".

```
let str1 = "Hello World";
console.log(str1.length);
```

Output:

```
11
```

Solution 2: toUpperCase()

Task: Convert the string "typescript" to upper case.

```
let str2 = "typescript";
console.log(str2.toUpperCase());
```

Output:

```
TYPESCRIPT
```

Solution 3: toLowerCase()

Task: Convert the string "TYPESCRIPT" to lower case.

```
let str3 = "TYPESCRIPT";
console.log(str3.toLowerCase());
```

Output:

```
typescript
```

Solution 4: charAt()

Task: Print the character present at index 2 of the string "Pavan".

```
let str4 = "Pavan";
console.log(str4.charAt(2));   // P=0, a=1, v=2
```

Output:

```
v
```

Solution 5: indexOf()

Task: Find the index where the word "coding" starts in the string "I love coding in TypeScript".

```
let str5 = "I love coding in TypeScript";
console.log(str5.indexOf("coding"));
```

Output:

```
7
```

Solution 6: includes()

Task: Check whether the string "Beginner TypeScript Guide" contains the word "Guide".

```
let str6 = "Beginner TypeScript Guide";
console.log(str6.includes("Guide"));
```

Output:

```
true
```

Solution 7: startsWith()

Task: Check whether the string "Welcome to TS" starts with "Welcome".

```
let str7 = "Welcome to TS";
console.log(str7.startsWith("Welcome"));
```

Output:

```
true
```

Solution 8: endsWith()

Task: Check whether the string "Learning TypeScript" ends with "Script".

```
let str8 = "Learning TypeScript";
console.log(str8.endsWith("Script"));
```

Output:

```
true
```

Solution 9: replace()

Task: In the string "I love JavaScript", replace "JavaScript" with "TypeScript".

```
let str9 = "I love JavaScript";
console.log(str9.replace("JavaScript", "TypeScript"));
```

Output:

```
I love TypeScript
```

Solution 10: trim() and split()

Task: Remove the extra spaces from the string " Hello World from TypeScript " and split it into words.

```
let str10 = "  Hello World from TypeScript  ";
let words = str10.trim().split(" ");  // first trim(), then split()
console.log(words);
```

Output:

```
[ 'Hello', 'World', 'from', 'TypeScript' ]
```